

Data Cleaning

```
[5]: import pandas as pd
df=pd.read_csv("online_learning_2026.csv")
print(df.head(10))
```

	student_id	age	gender	country	education_level	course_category \
0	1	54	Male	UK	Master's	AI/ML
1	2	44	Male	Canada	Bachelor's	Web Development
2	3	30	Male	Mexico	Bachelor's	Data Science
3	4	23	Male	Brazil	Bachelor's	AI/ML
4	5	36	Male	China	Bachelor's	Cloud Computing
5	6	54	Female	India	Master's	Cloud Computing
6	7	34	Male	Japan	High School	AI/ML
7	8	38	Female	Brazil	PhD	AI/ML
8	9	26	Male	China	Master's	AI/ML
9	10	26	Female	Australia	Bachelor's	Cloud Computing

	platform	course_duration_weeks	hours_per_week	quiz_scores_avg \
0	Codecademy	41	4	29.31
1	edX	29	23	14.78
2	LinkedIn Learning	41	27	12.98
3	LinkedIn Learning	36	18	63.46
4	Codecademy	5	18	32.80
5	Coursera	39	22	27.50
6	Udemy	26	9	59.21
7	Udemy	39	3	2.57
8	Codecademy	11	7	24.62
9	DataCamp	42	26	34.05

	...	previous_courses_completed	financial_aid	course_price_usd	\
0	...	2	No	124.32	
1	...	12	No	155.13	
2	...	1	Yes	40.85	
3	...	12	No	14.99	
4	...	20	No	73.08	
5	...	14	Yes	141.80	
6	...	7	No	137.22	
7	...	11	No	191.02	
8	...	7	Yes	126.00	
9	...	18	No	82.26	

	self_motivation_score	time_management_score	learning_disability	\
0	4	3	No	
1	9	1	No	
2	10	7	No	
3	2	10	No	
4	10	10	No	
5	10	10	No	
6	6	7	Yes	
7	4	6	No	
8	8	9	No	
9	4	7	No	

	employment_status	completion_status	final_grade	satisfaction_rating
0	Employed	Completed	F	3.3
1	Employed	Completed	F	2.7
2	Employed	Completed	F	2.8
3	Student	Completed	D	4.1
4	Employed	Completed	F	3.8
5	Unemployed	Completed	F	3.2
6	Employed	Completed	F	4.1
7	Unemployed	Dropped	F	1.6
8	Student	In Progress	F	2.6
9	Student	Dropped	F	1.3

```
: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 4000 entries, 0 to 3999
```

```
Data columns (total 25 columns):
```

#	Column	Non-Null Count	Dtype
0	student_id	4000 non-null	int64
1	age	4000 non-null	int64
2	gender	4000 non-null	object
3	country	4000 non-null	object
4	education_level	4000 non-null	object
5	course_category	4000 non-null	object
6	platform	4000 non-null	object
7	course_duration_weeks	4000 non-null	int64
8	hours_per_week	4000 non-null	int64
9	quiz_scores_avg	4000 non-null	float64
10	assignment_completion_rate	4000 non-null	float64
11	forum_posts	4000 non-null	int64
12	videos_watched_pct	4000 non-null	float64
13	device	4000 non-null	object
14	internet_speed	4000 non-null	object
15	previous_courses_completed	4000 non-null	int64
16	financial_aid	4000 non-null	object
17	course_price_usd	4000 non-null	float64
18	self_motivation_score	4000 non-null	int64
19	time_management_score	4000 non-null	int64
20	learning_disability	4000 non-null	object
21	employment_status	4000 non-null	object
22	completion_status	4000 non-null	object
23	final_grade	4000 non-null	object
24	satisfaction_rating	4000 non-null	float64

```
dtypes: float64(5), int64(8), object(12)
```

```
memory usage: 781.4+ KB
```

	employment_status	completion_status	final_grade	satisfaction_rating
0	Employed	Completed	F	3.3
1	Employed	Completed	F	2.7
2	Employed	Completed	F	2.8
3	Student	Completed	D	4.1
4	Employed	Completed	F	3.8
5	Unemployed	Completed	F	3.2
6	Employed	Completed	F	4.1
7	Unemployed	Dropped	F	1.6
8	Student	In Progress	F	2.6
9	Student	Dropped	F	1.3

[10 rows x 25 columns]

```
#check current column names
print(df.columns)
```

```
Index(['student_id', 'age', 'gender', 'country', 'education_level',
      'course_category', 'platform', 'course_duration_weeks',
      'hours_per_week', 'quiz_scores_avg', 'assignment_completion_rate',
      'forum_posts', 'videos_watched_pct', 'device', 'internet_speed',
      'previous_courses_completed', 'financial_aid', 'course_price_usd',
      'self_motivation_score', 'time_management_score', 'learning_disability',
      'employment_status', 'completion_status', 'final_grade',
      'satisfaction_rating'],
      dtype='object')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4000 entries, 0 to 3999
Data columns (total 25 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   student_id            4000 non-null   int64
 1   age                   4000 non-null   int64
 2   gender                4000 non-null   object
 3   country               4000 non-null   object
```



```
#Rename columns
df.rename(columns={
    'student_id':'Student_ID',
    'country':'Country_Name'
},inplace=True)
print(df.columns)
```

```
Index(['Student_ID', 'age', 'gender', 'Country_Name', 'education_level',
       'course_category', 'platform', 'course_duration_weeks',
       'hours_per_week', 'quiz_scores_avg', 'assignment_completion_rate',
       'forum_posts', 'videos_watched_pct', 'device', 'internet_speed',
       'previous_courses_completed', 'financial_aid', 'course_price_usd',
       'self_motivation_score', 'time_management_score', 'learning_disability',
       'employment_status', 'completion_status', 'final_grade',
       'satisfaction_rating'],
      dtype='object')
```

```
#Check Data Types.-
print(df.dtypes)
```

Student_ID	int64
age	int64
gender	object
Country_Name	object
education_level	object
course_category	object
platform	object
course_duration_weeks	int64
hours_per_week	int64
quiz_scores_avg	float64
assignment_completion_rate	float64
forum_posts	int64
videos_watched_pct	float64
device	object
internet_speed	object
previous_courses_completed	int64

```
course_price_usd      float64
self_motivation_score  int64
time_management_score  int64
learning_disability    object
employment_status      object
completion_status      object
final_grade            object
satisfaction_rating    float64
dtype: object
```

```
] duplicates=df.duplicated() #if it have dup it return true otherwise it returns false
print(duplicates)
```

```
0      False
1      False
2      False
3      False
4      False
...
3995   False
3996   False
3997   False
3998   False
3999   False
Length: 4000, dtype: bool
```

```
] df.isnull().sum()
```

```
] Student_ID      0
age               0
gender            0
Country_Name      0
education_level   0
course_category   0
platform          0
course_duration_weeks  0
hours_per_week    0
```

device	object
internet_speed	object
previous_courses_completed	int64
financial_aid	object
course_price_usd	float64
self_motivation_score	int64
time_management_score	int64
learning_disability	object
employment_status	object
completion_status	object
final_grade	object
satisfaction_rating	float64
dtype:	object

```

9]: #Convert Student_ID to String
df['Student_ID']=df['Student_ID'].astype(str)
print(df.dtypes)

```

Student_ID	object
age	int64
gender	object
Country_Name	object
education_level	object
course_category	object
platform	object
course_duration_weeks	int64
hours_per_week	int64
quiz_scores_avg	float64
assignment_completion_rate	float64
forum_posts	int64
videos_watched_pct	float64
device	object
internet_speed	object
previous_courses_completed	int64
financial_aid	object
course_price_usd	float64
self_motivation_score	int64

length: 4000, dtype: bool

```
df.isnull().sum()
```

Student_ID	0
age	0
gender	0
Country_Name	0
education_level	0
course_category	0
platform	0
course_duration_weeks	0
hours_per_week	0
quiz_scores_avg	0
assignment_completion_rate	0
forum_posts	0
videos_watched_pct	0
device	0
internet_speed	0
previous_courses_completed	0
financial_aid	0
course_price_usd	0
self_motivation_score	0
time_management_score	0
learning_disability	0
employment_status	0
completion_status	0
final_grade	0
satisfaction_rating	0
dtype:	int64

```
#count duplicates:
```

```
print(df.duplicated().sum())
```

0

#5.Dropping duplicate records

#Before

```
print(df.shape)
```

#Remove duplicates

```
df.drop_duplicates(inplace=True)
```

#After

```
print(df.shape)
```

```
(4000, 25)
```

```
(4000, 25)
```

#6.String operations

#string methods work only on text col

#Remove extra spaces

```
df['Country_Name']=df['Country_Name'].str.strip() #using strip to remove spaces
```

#convert to uppercase

```
df['Country_Name']=df['Country_Name'].str.upper()
```

```
print(df['Country_Name'].head())
```

#Convert to title case

```
df['Country_Name']=df['Country_Name'].str.title() #1st letter only caps
```

```
print(df['Country_Name'].head())
```

```
0      UK
```

```
1  CANADA
```

```
2  MEXICO
```

```
3  BRAZIL
```

```
4  CHINA
```

```
Name: Country_Name, dtype: object
```

```
0      Uk
```

```
1  Canada
```

```
2  Mexico
```

```
3  Brazil
```

```
4  China
```

```
Name: Country_Name, dtype: object
```

```
df['course_category']=df['course_category'].replace({'AI/ML':'Artificial Intelligence and Machine Learning',
                                                    'UX Design':'User Experience'})

print(df['course_category'].head()) #for multi values use dict
```

```
0    Artificial Intelligence and Machine Learning
1                                Web Development
2                                Data Science
3    Artificial Intelligence and Machine Learning
4                                Cloud Computing
Name: course_category, dtype: object
```

```
#String length
df['Name_Length']=df['Country_Name'].str.len() #add new col and find len
print(df[['Country_Name','Name_Length']].head())
```

```
Country_Name  Name_Length
0          Uk            2
1        Canada            6
2        Mexico            6
3        Brazil            6
4         China            5
```

```
#Extract first character
df['First_Letter']=df['Country_Name'].str[0] #access val

print(df[['Country_Name','First_Letter']].head())
```

	Country_Name	First_Letter
0	Uk	U
1	Canada	C
2	Mexico	M
3	Brazil	B
4	China	C

```
#Unique values for country name
```

```
print(df['Country_Name'].unique())
```

```
['Uk' 'Canada' 'Mexico' 'Brazil' 'China' 'India' 'Japan' 'Australia'
 'France' 'Germany' 'Usa' 'Nigeria']
```

Data Preprocessing

```
import pandas as pd
import seaborn as sns
```

```
#Load titanic dataset
```

```
df=pd.read_csv("online_learning_2026.csv")
```

```
print(df.head())
```

```
print(df.shape)
```

	student_id	age	gender	country	education_level	course_category
0	1	54	Male	UK	Master's	AI/ML
1	2	44	Male	Canada	Bachelor's	Web Development
2	3	30	Male	Mexico	Bachelor's	Data Science
3	4	23	Male	Brazil	Bachelor's	AI/ML
4	5	36	Male	China	Bachelor's	Cloud Computing

	platform	course_duration_weeks	hours_per_week	quiz_scores_avg	\
0	Codecademy	41	4	29.31	
1	edX	29	23	14.78	
2	LinkedIn Learning	41	27	12.98	
3	LinkedIn Learning	36	18	63.46	
4	Codecademy	5	18	32.80	

	...	previous_courses_completed	financial_aid	course_price_usd	\
0	...	2	No	124.32	
1	...	12	No	155.13	
2	...	1	Yes	40.85	
3	...	12	No	14.99	
4	...	20	No	73.08	

	self_motivation_score	time_management_score	learning_disability	\
0	4	3	No	
1	9	1	No	
2	10	7	No	
3	2	10	No	
4	10	10	No	

	employment_status	completion_status	final_grade	satisfaction_rating
0	Employed	Completed	F	3.3
1	Employed	Completed	F	2.7
2	Employed	Completed	F	2.8
3	Student	Completed	D	4.1
4	Employed	Completed	F	3.8

[5 rows x 25 columns]
(4000, 25)

```
Country_Name
Australia    2
Name: count, dtype: int64
```

```
print(df['Country_Name'].unique()) #again print previous val
```

```
[nan 'Australia']
```

```
df.loc[0,'age']=240
df.loc[1,'age']=300
#Detect Unusual values
df['age'].describe()
```

```
count    4000.000000
mean      35.838000
std       12.697857
min       16.000000
25%       26.000000
50%       36.000000
75%       46.000000
max       300.000000
Name: age, dtype: float64
```

```
#Fix Unit Mismatch
#Convert months to years
df.loc[df['age']>120,'age']=(
    df.loc[df['age']>120,'age']/12 #above 120 div by 12 and convert months to yrs.
)
#Verify
df['age'].describe()
```



```
#create new names
df.loc[11,'Country_Name']='australia'
df.loc[26,'Country_Name']='AUSTRALIA' #row,col change these 3 val and print all so first print these and bal
print(df['Country_Name'].unique())
```

```
[nan 'australia' 'AUSTRALIA']
```

```
#It change all words first letter in capital and balance small letter
df['Country_Name']=df['Country_Name'].str.title()
print(df['Country_Name'].unique())
```

```
[nan 'Australia']
```

```
print(df['Country_Name'].isnull().sum()) #total nll val
```

```
3998
```

```
#Typos
#Spelling mistakes create unnecessary categories.
#create type
df.loc[11,'Country_Name']='Ausstralia' #we create mistakes
print(df['Country_Name'].value_counts()) #mistake words also add
```

```
Country_Name
Ausstralia    1
Australia     1
Name: count, dtype: int64
```

```
#Fix types
df['Country_Name']=df['Country_Name'].replace({
    'Ausstralia':'Australia',
})
```

```
#Verify
print(df['Country_Name'].value_counts()) #back to form
```

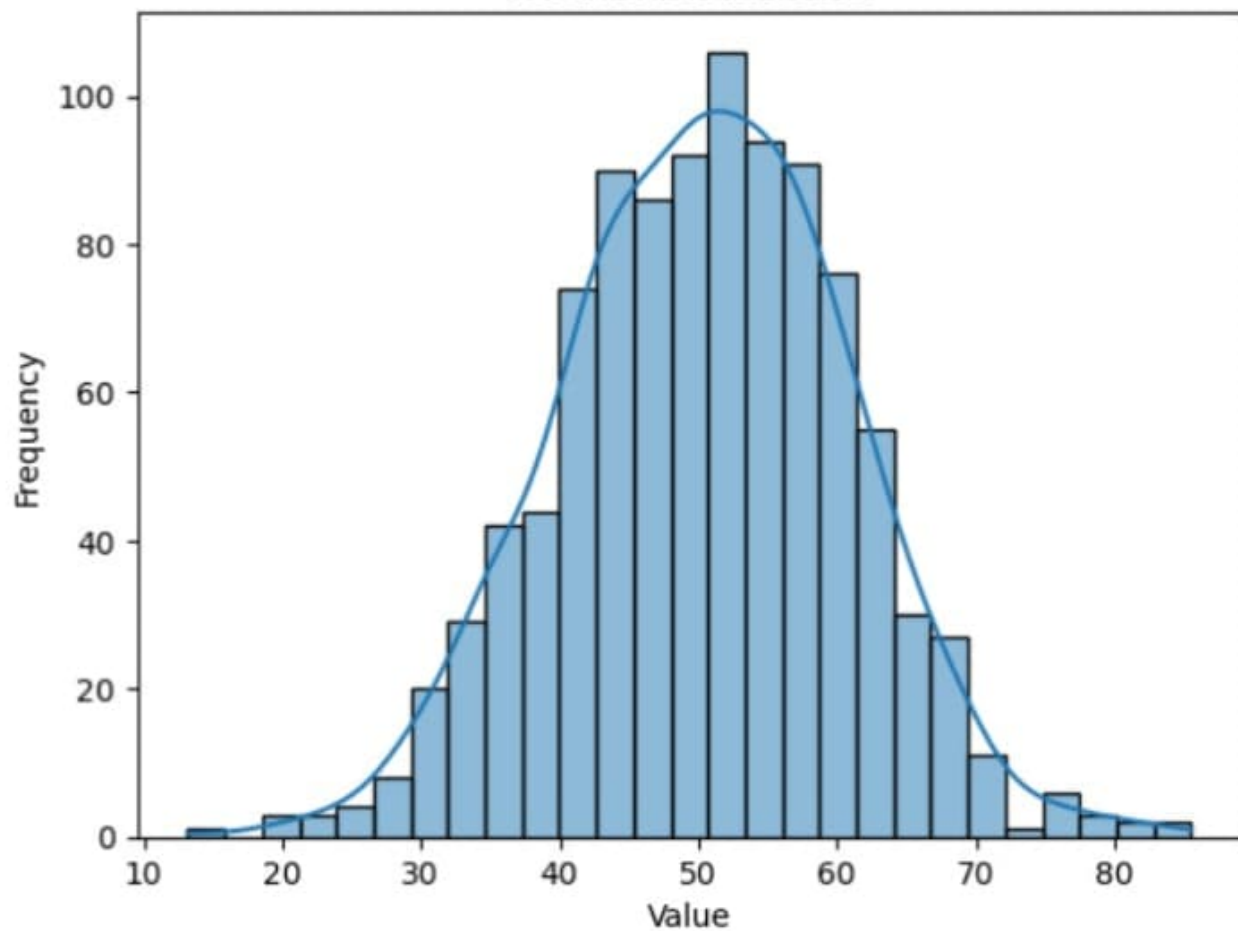
```
count    4000.00000
mean      35.71425
std       11.55160
min       16.00000
25%       26.00000
50%       36.00000
75%       46.00000
max       55.00000
Name: age, dtype: float64
```

#check:

```
df.isnull().mean()
```

```
student_id      0.0000
age             0.0000
gender          0.0000
country         0.0000
education_level 0.0000
course_category 0.0000
platform        0.0000
course_duration_weeks 0.0000
hours_per_week  0.0000
quiz_scores_avg 0.0000
assignment_completion_rate 0.0000
forum_posts     0.0000
videos_watched_pct 0.0000
device          0.0000
internet_speed  0.0000
previous_courses_completed 0.0000
financial_aid    0.0000
course_price_usd 0.0000
self_motivation_score 0.0000
time_management_score 0.0000
learning_disability 0.0000
employment_status 0.0000
...            ...
```

Normal Distribution



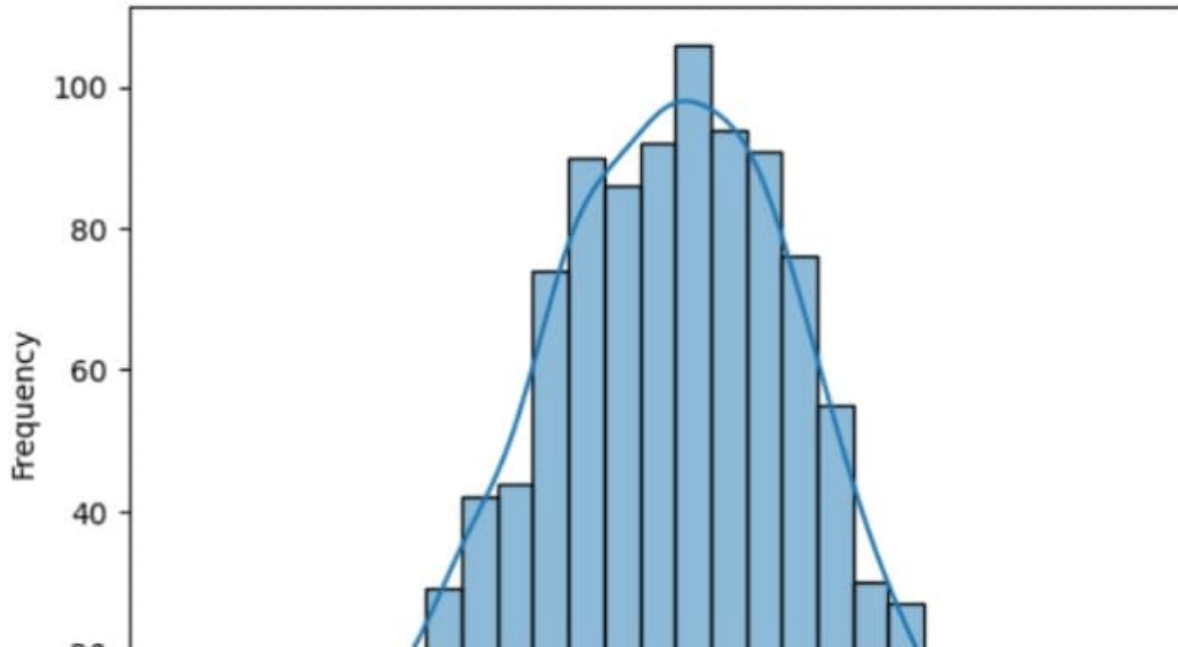
Data Visualization

```
#Normal distribution
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

data = np.random.normal(loc=50, scale=10, size=1000)

sns.histplot(data, kde=True)
plt.title("Normal Distribution")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```

Normal Distribution

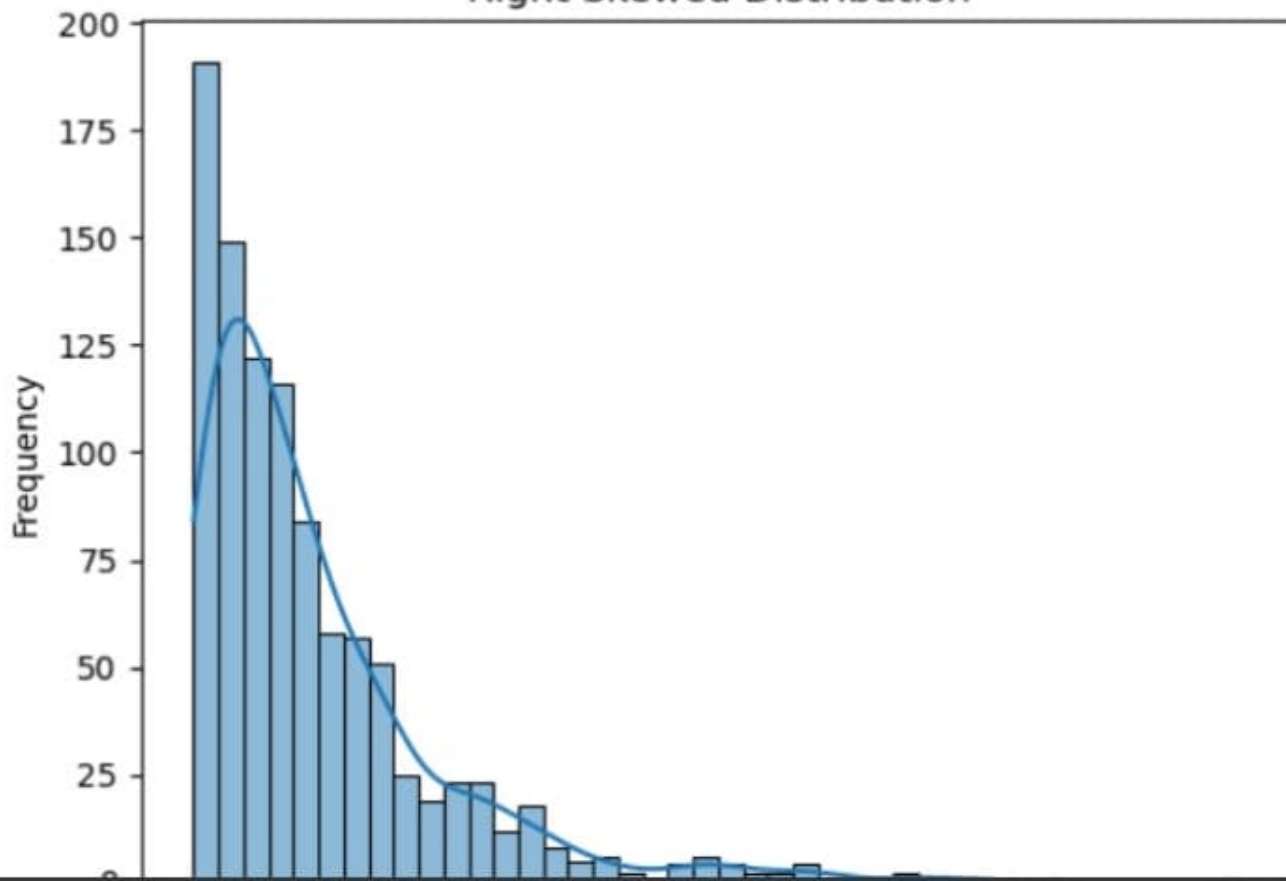


```
1: #Right skewed distribution
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

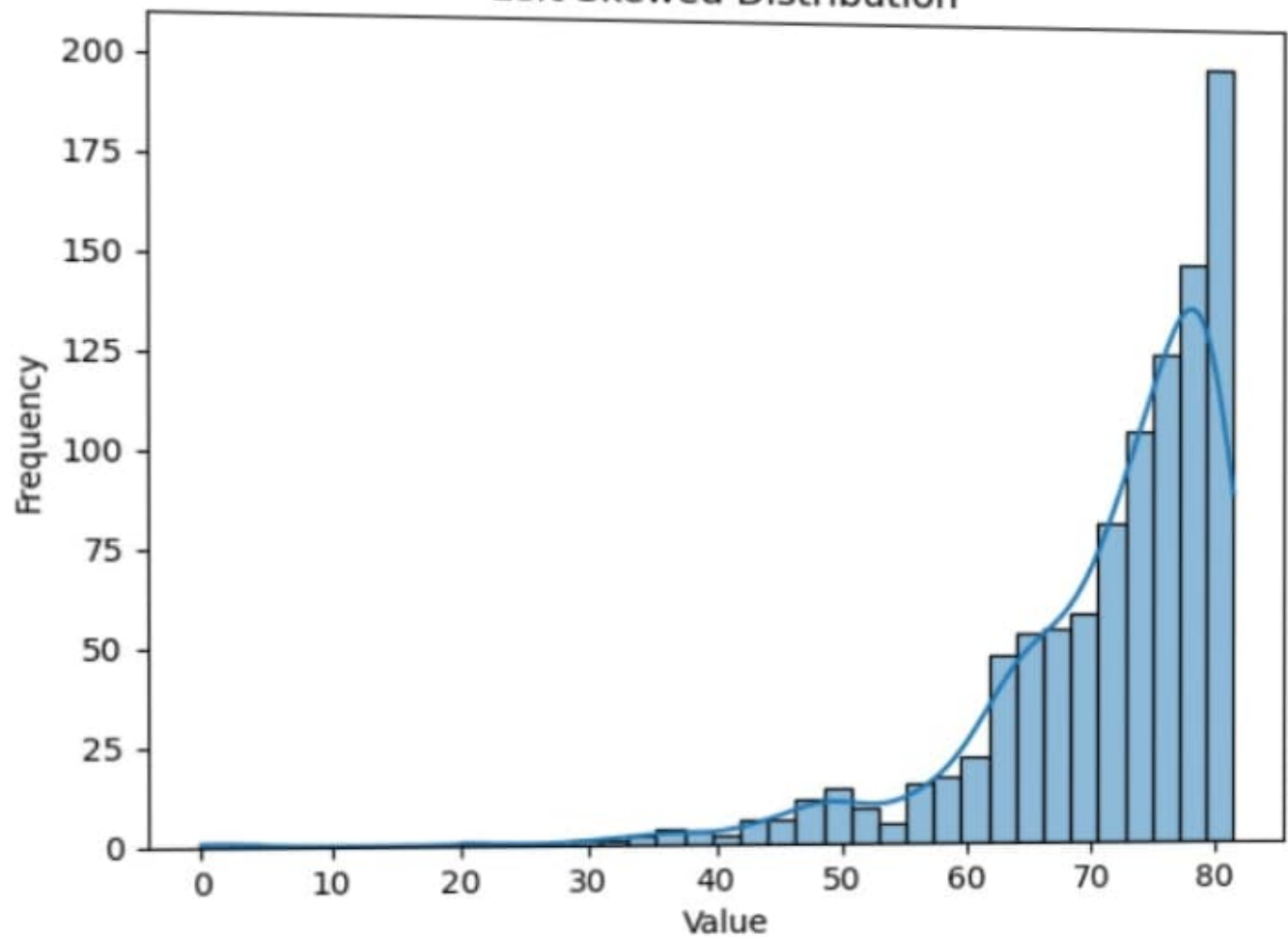
data = np.random.exponential(scale=10, size=1000)

sns.histplot(data, kde=True)
plt.title("Right Skewed Distribution")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```

Right Skewed Distribution



Left Skewed Distribution



```
#Left skewed distribution
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

data = -np.random.exponential(scale=10, size=1000)
data = data + abs(data.min())

sns.histplot(data, kde=True)
plt.title("Left Skewed Distribution")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```

: *#1.Pearson Correlation*

```
import pandas as pd
```

```
df = pd.read_csv("online_learning_2026.csv")
```

```
corr = df['hours_per_week'].corr(df['quiz_scores_avg'])
```

```
print("Pearson Correlation:", corr)
```

Pearson Correlation: -0.0025324416051423497

: *#2.Spearman Correlation*

```
import pandas as pd
```

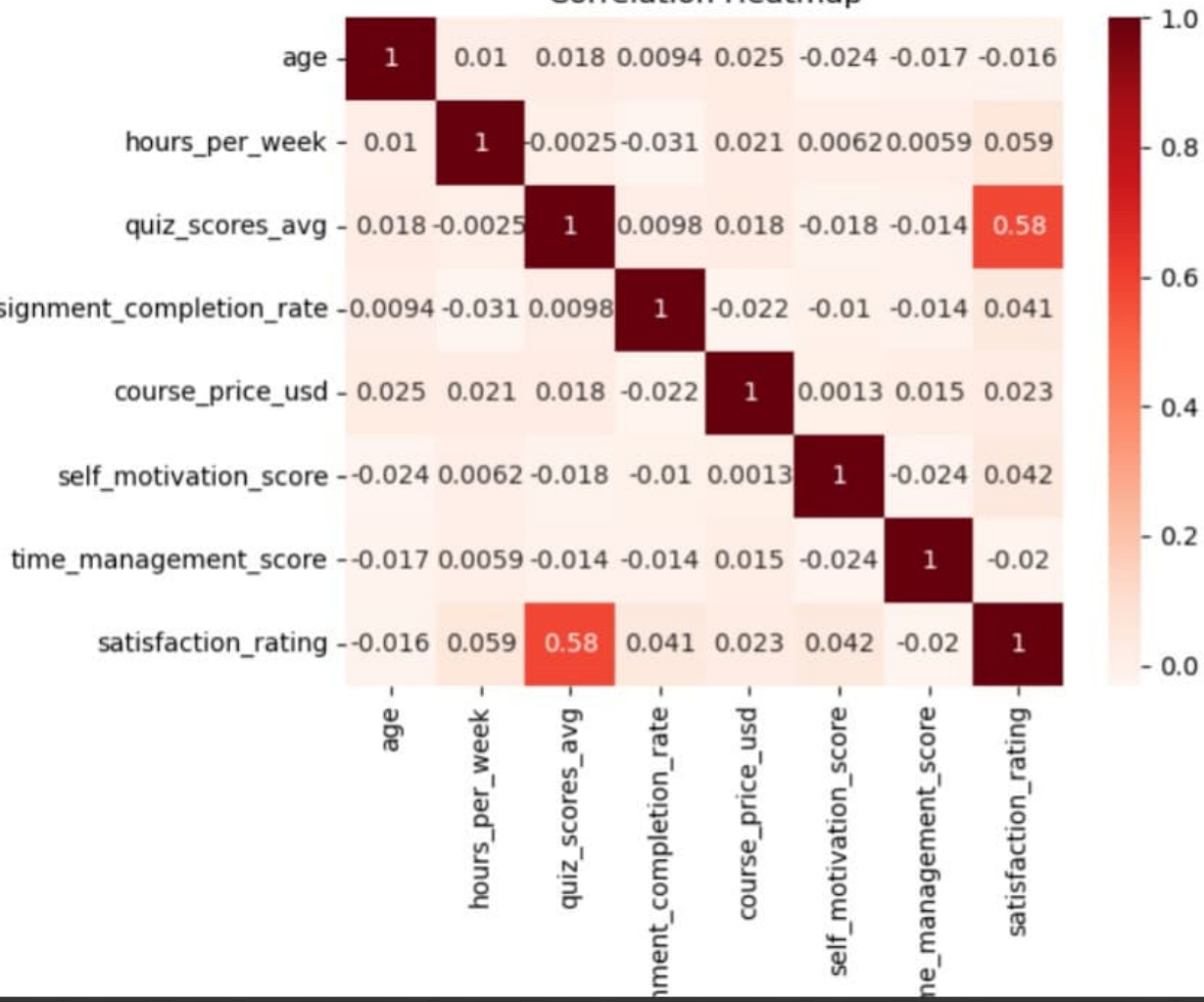
```
df = pd.read_csv("online_learning_2026.csv")
```

```
corr = df['self_motivation_score'].corr(  
    df['time_management_score'],  
    method='spearman'  
)
```

```
print("Spearman Correlation:", corr)
```

Spearman Correlation: -0.02383808101802208

Correlation Heatmap



#3. Correlation Heatmap

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

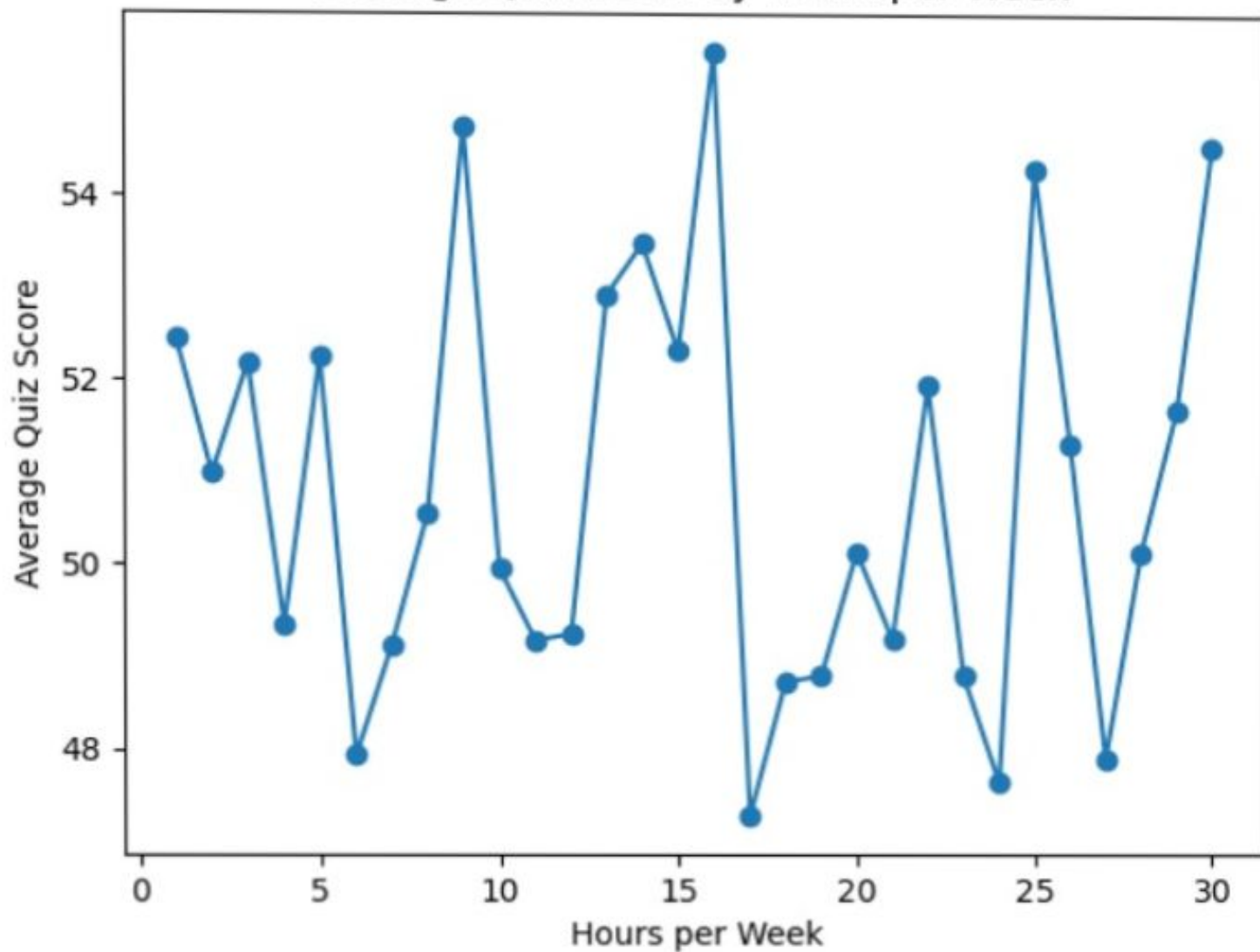
df = pd.read_csv("online_learning_2026.csv")

# Select numeric columns
data = df[['age',
            'hours_per_week',
            'quiz_scores_avg',
            'assignment_completion_rate',
            'course_price_usd',
            'self_motivation_score',
            'time_management_score',
            'satisfaction_rating']]

sns.heatmap(
    data.corr(),
    annot=True,
    cmap='Reds'
)

plt.title("Correlation Heatmap")
plt.show()
```


Average Quiz Score by Hours per Week



```
#Line chart
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("online_learning_2026.csv")

# Calculate average quiz score for each study hour
avg_score = df.groupby("hours_per_week")["quiz_scores_avg"].mean()

plt.plot(avg_score.index, avg_score.values, marker='o')

plt.title("Average Quiz Score by Hours per Week")
plt.xlabel("Hours per Week")
plt.ylabel("Average Quiz Score")

plt.show()
```

```
#Bar chart

import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("online_learning_2026.csv")

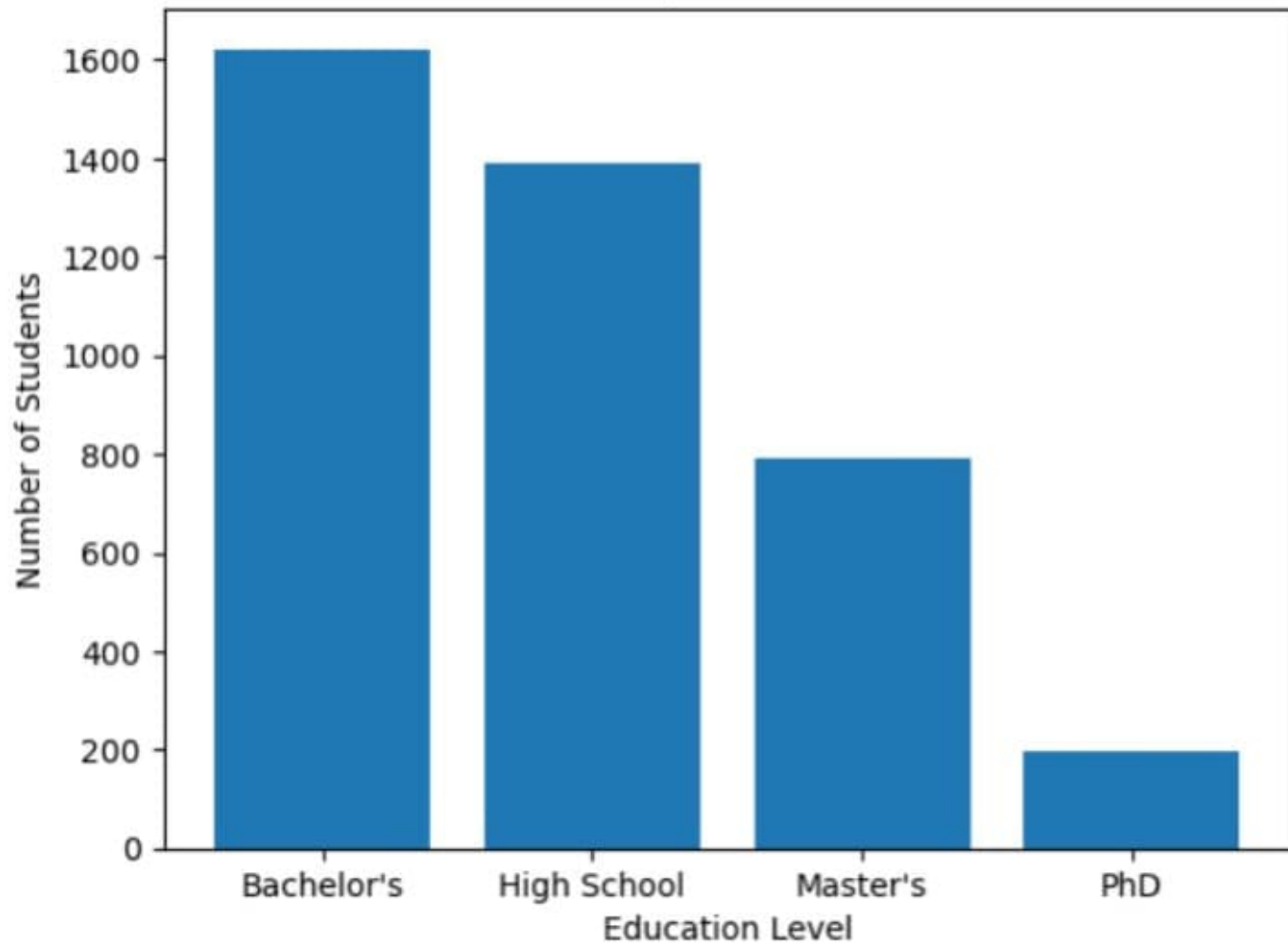
# Count students in each education level
education_count = df['education_level'].value_counts()

plt.bar(education_count.index, education_count.values)

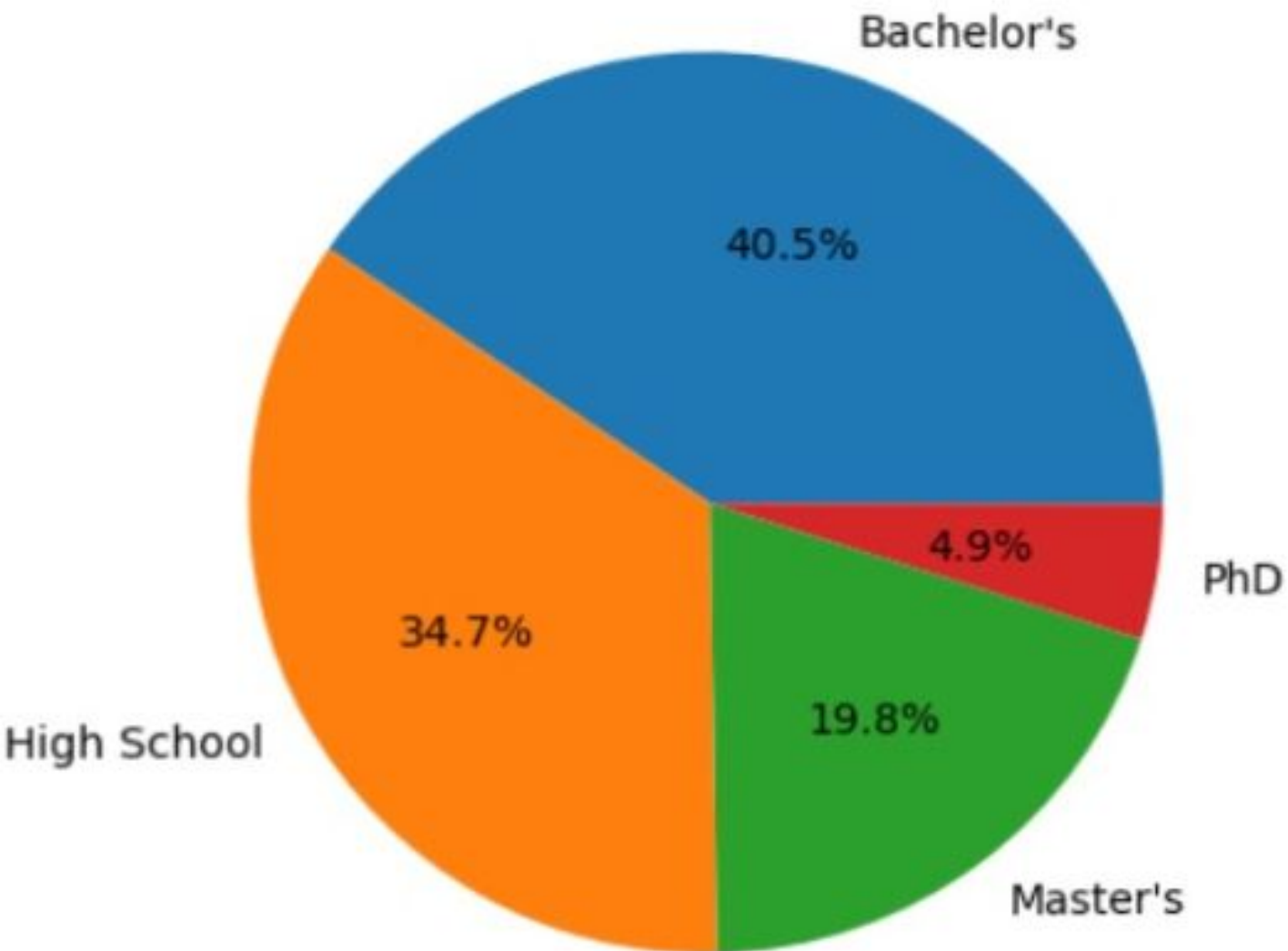
plt.title("Students by Education Level")
plt.xlabel("Education Level")
plt.ylabel("Number of Students")

plt.show()
```

Students by Education Level



Students by Education Level



Analysis

- This chart shows how many students are in each education level.
- Some education levels have more students, while some have fewer students.

Business Insights

- Focus more on the education level with the most students.
- Provide offers and courses to attract students from other education levels.
- This helps increase student enrollment and platform growth.

```
#Pie chart
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("online_learning_2026.csv")

education_count = df['education_level'].value_counts()

plt.pie(
    education_count.values,
    labels=education_count.index,
    autopct='%1.1f%%'
)

plt.title("Students by Education Level")
plt.show()
```


Analysis

- The pie chart shows the percentage of students in each education level.
- Some education levels have a larger share than others.

Business Insights

- Focus more on the education level with the highest percentage.
- Create offers and courses to attract other education groups.

#Scatter plot

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("online_learning_2026.csv")

# Take only first 10 rows
df = df.head(10)

plt.scatter(df['hours_per_week'], df['course_price_usd'])

plt.title("Hours per Week vs Course Price")
plt.xlabel("Hours per Week")
plt.ylabel("Course Price (USD)")

plt.show()
```

Analysis:

- The chart shows the relationship between hours per week and course price.

Business Insights:

- It helps understand course pricing and study time patterns.

```
9]: #Area chart
#Purpose:Shows cumulaive totals over time.
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("online_learning_2026.csv")

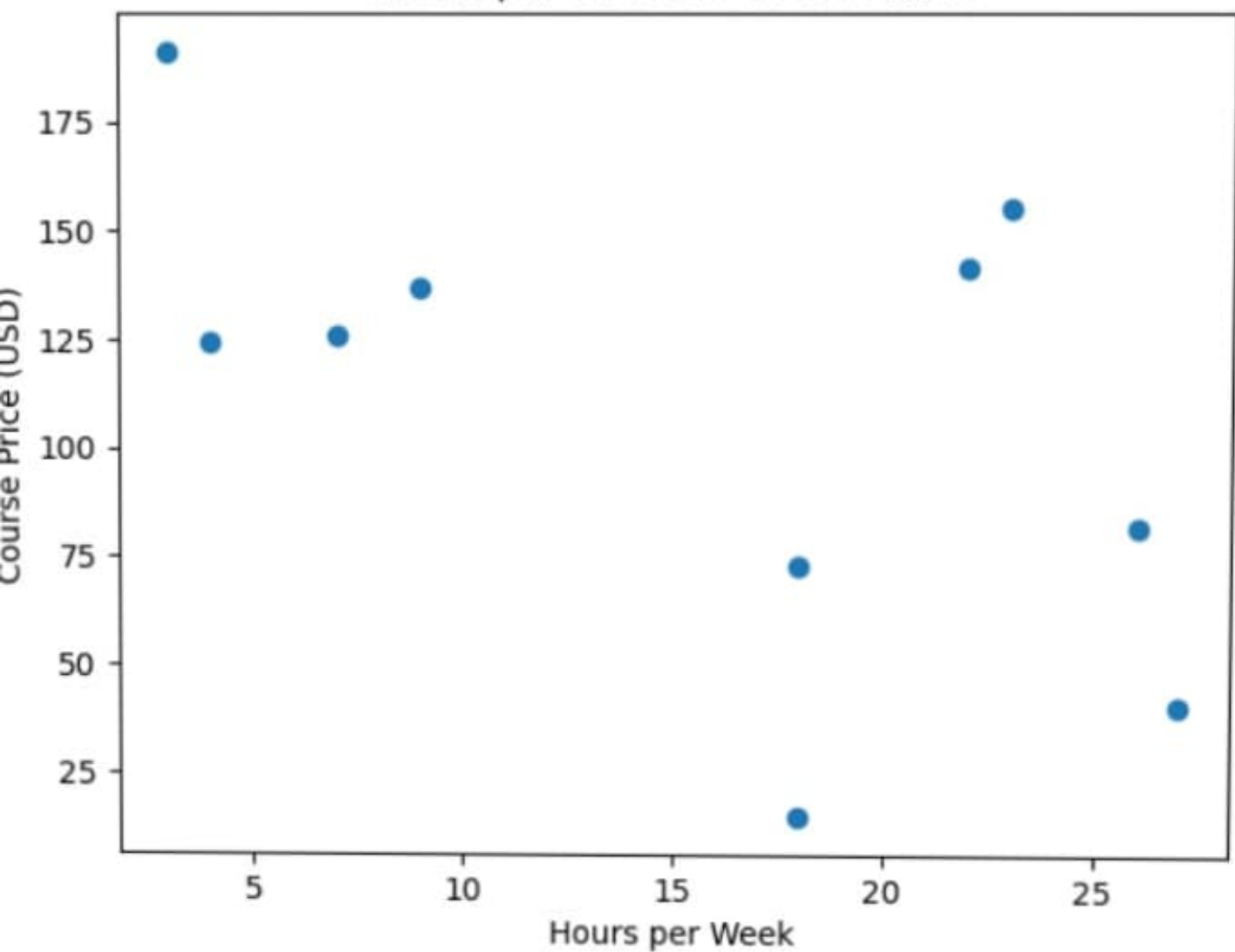
# Group by hours_per_week and calculate average quiz score
data = df.groupby("hours_per_week")["quiz_scores_avg"].mean()

plt.fill_between(data.index, data.values)

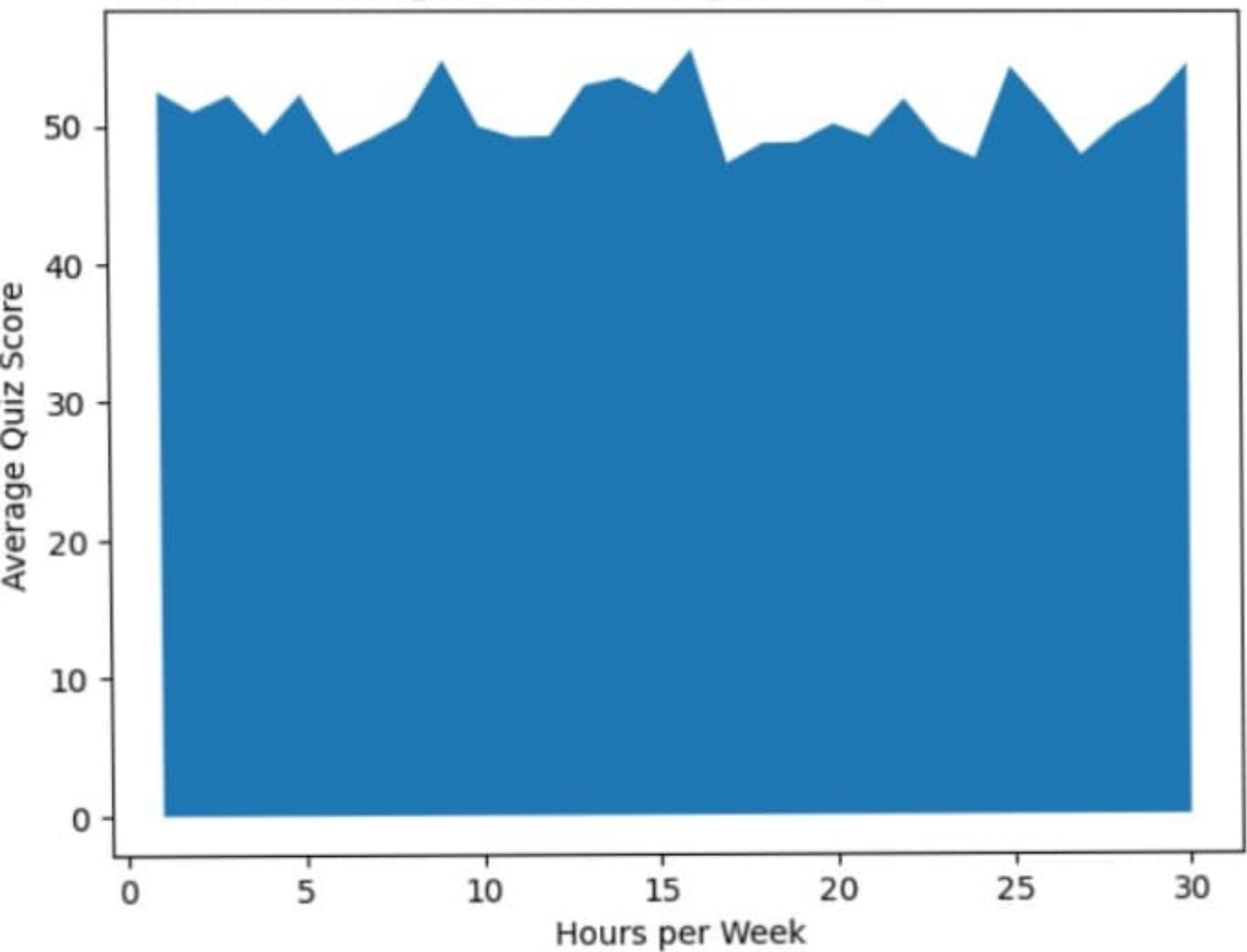
plt.title("Average Quiz Score by Hours per Week")
plt.xlabel("Hours per Week")
plt.ylabel("Average Quiz Score")

plt.show()
```

Hours per Week vs Course Price



Average Quiz Score by Hours per Week



Analysis:

- The chart shows average quiz scores based on study hours.

Business Insights:

- It helps understand the impact of study time on student performance.

```
[113]: ### Analysis  
  
* Shows the distribution of satisfaction ratings.  
* Helps identify outliers.  
  
### Business Insights  
  
* Helps measure student satisfaction.  
* Outliers can be used to improve course quality.
```

Analysis

- Shows the distribution of satisfaction ratings.
- Helps identify outliers.

Business Insights

- Helps measure student satisfaction.
- Outliers can be used to improve course quality.

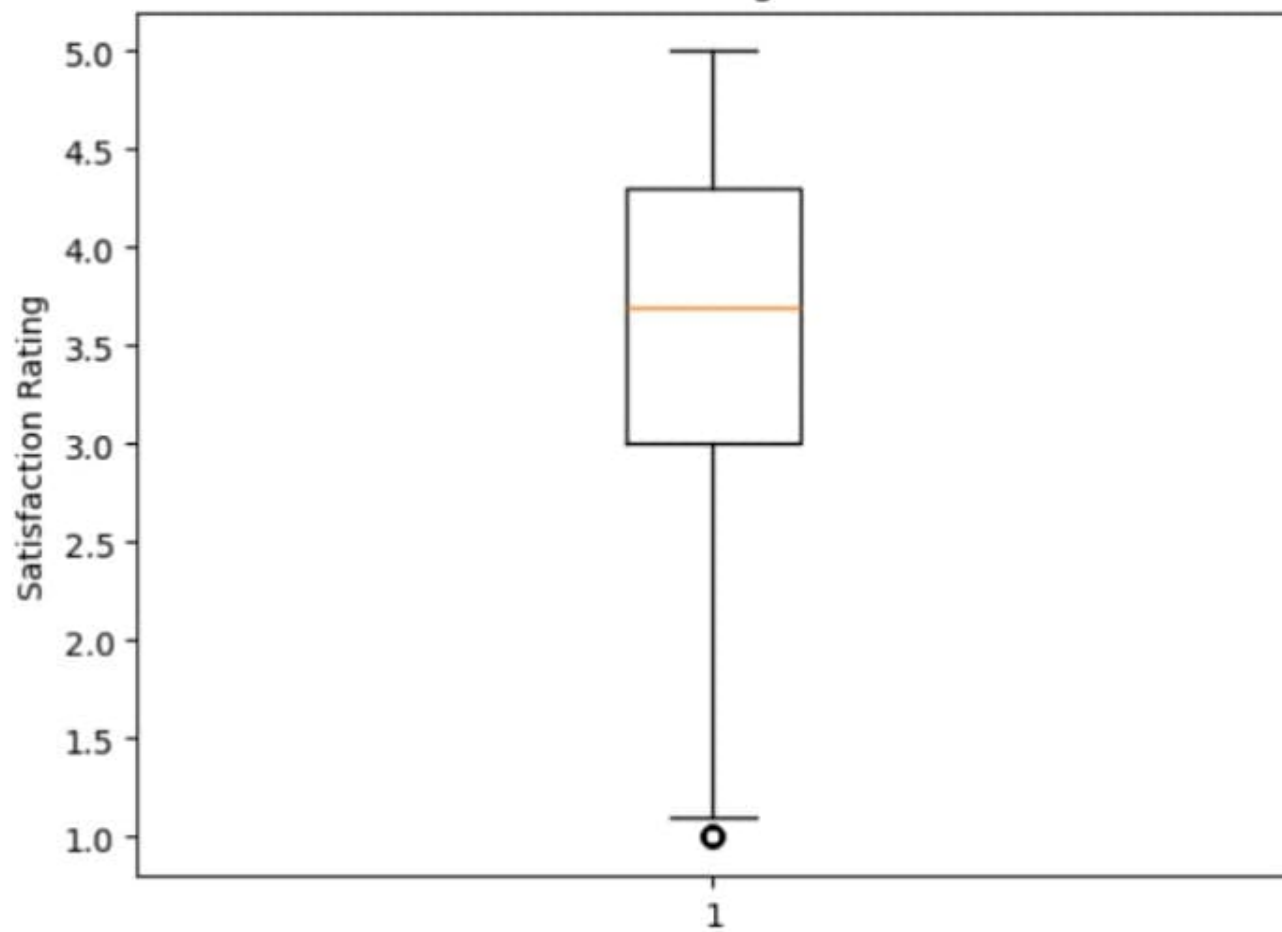
]:

```
#Histogram
sns.histplot(
    data=df,
    x="age",
    bins=10
)

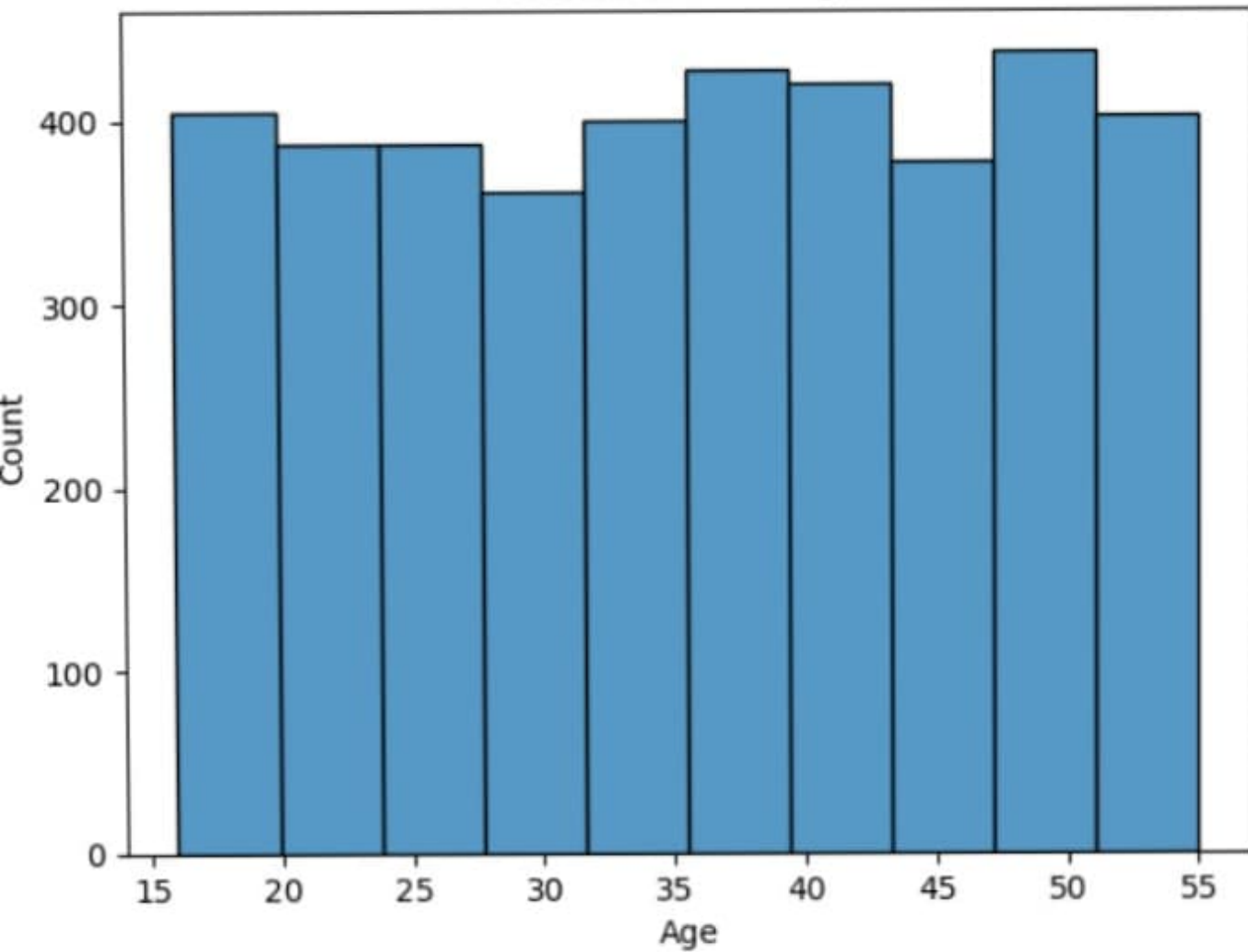
plt.title("Distribution of Age")
plt.xlabel("Age")
plt.ylabel("Count")

plt.show()
```


Satisfaction Rating Distribution



Distribution of Age

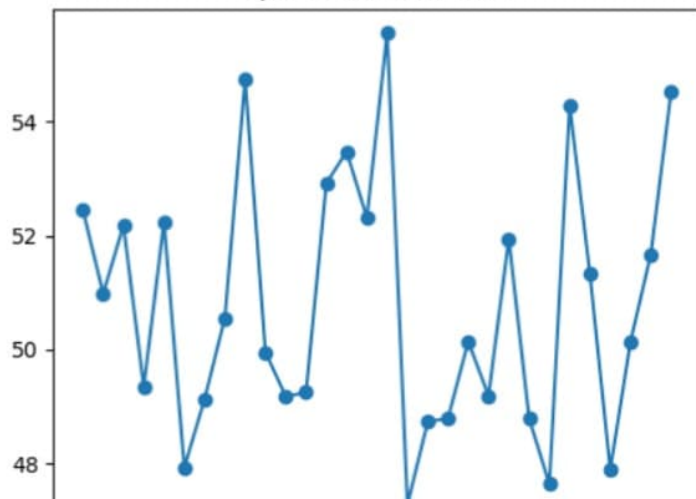


```
# 3. Scatter Plot
plt.subplot(2,2,3)
plt.scatter(df['hours_per_week'], df['quiz_scores_avg'])
plt.title("Hours vs Quiz Score")
```

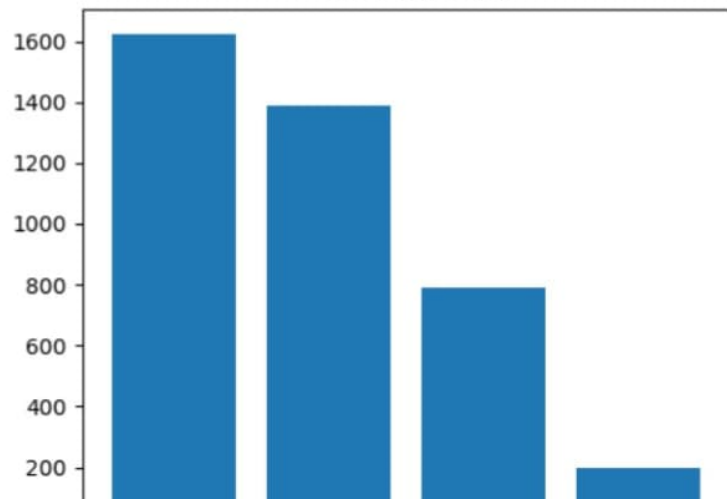
```
# 4. Pie Chart
plt.subplot(2,2,4)
platform = df['platform'].value_counts()
plt.pie(platform.values,
        labels=platform.index,
        autopct='%1.1f%%')
plt.title("Platform Distribution")

plt.tight_layout()
plt.show()
```

Quiz Score vs Hours



Education Level



Analysis

- Shows the distribution of student ages.
- Identifies the most common age group.

Business Insights

- Helps target the main age group.
- Supports better course and marketing planning.

```
#Four charts
import pandas as pd
import matplotlib.pyplot as plt

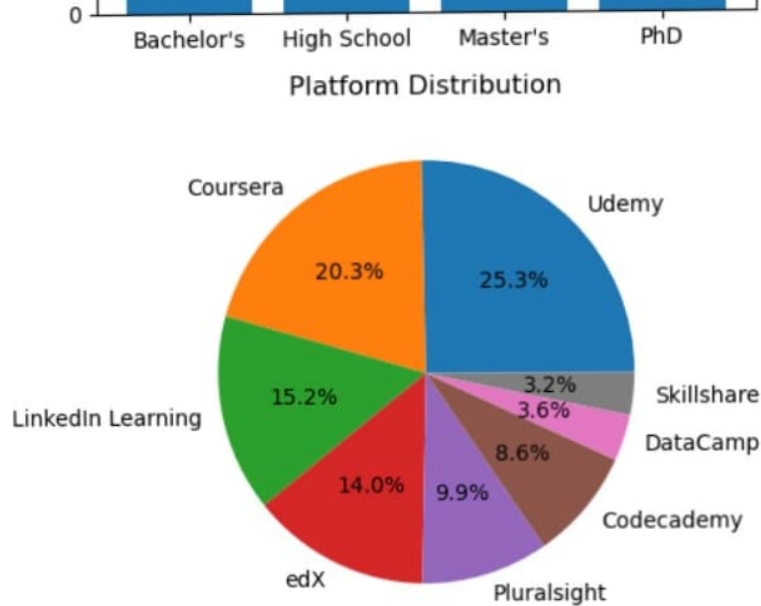
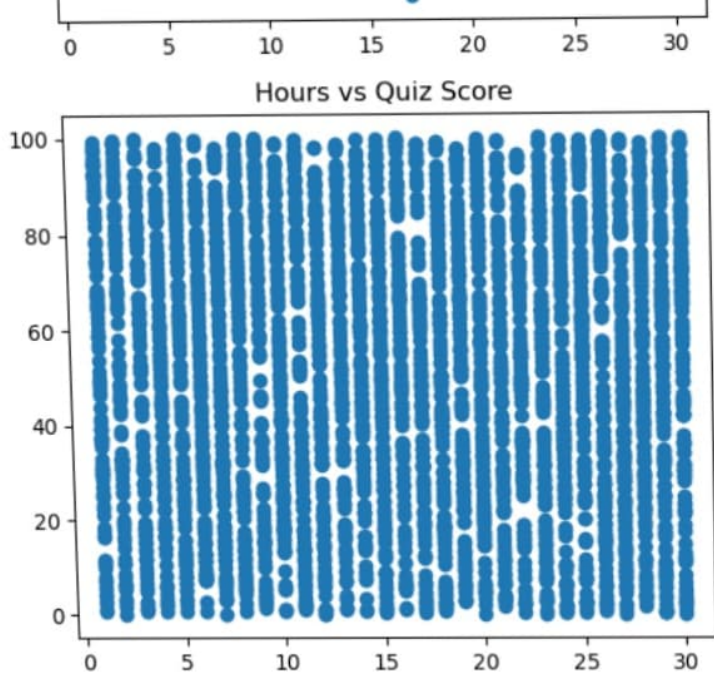
# Load dataset
df = pd.read_csv("online_learning_2026.csv")

plt.figure(figsize=(10,8))

# 1. Line Chart
plt.subplot(2,2,1)
line_data = df.groupby('hours_per_week')['quiz_scores_avg'].mean()
plt.plot(line_data.index, line_data.values, marker='o')
plt.title("Quiz Score vs Hours")

# 2. Bar Chart
plt.subplot(2,2,2)
edu = df['education_level'].value_counts()
plt.bar(edu.index, edu.values)
plt.title("Education Level")

# 3. Scatter Plot
plt.subplot(2,2,3)
```



Analysis

- The charts show quiz scores, education levels, study hours, and platform usage.
- They help understand student learning patterns and preferences.

Business Insights

- Identify the most used platform and education group.
- Understand how study hours affect quiz scores.
- Use these insights to improve courses and student engagement.

EDA

Univariate Analysis (One Variable)

Histogram

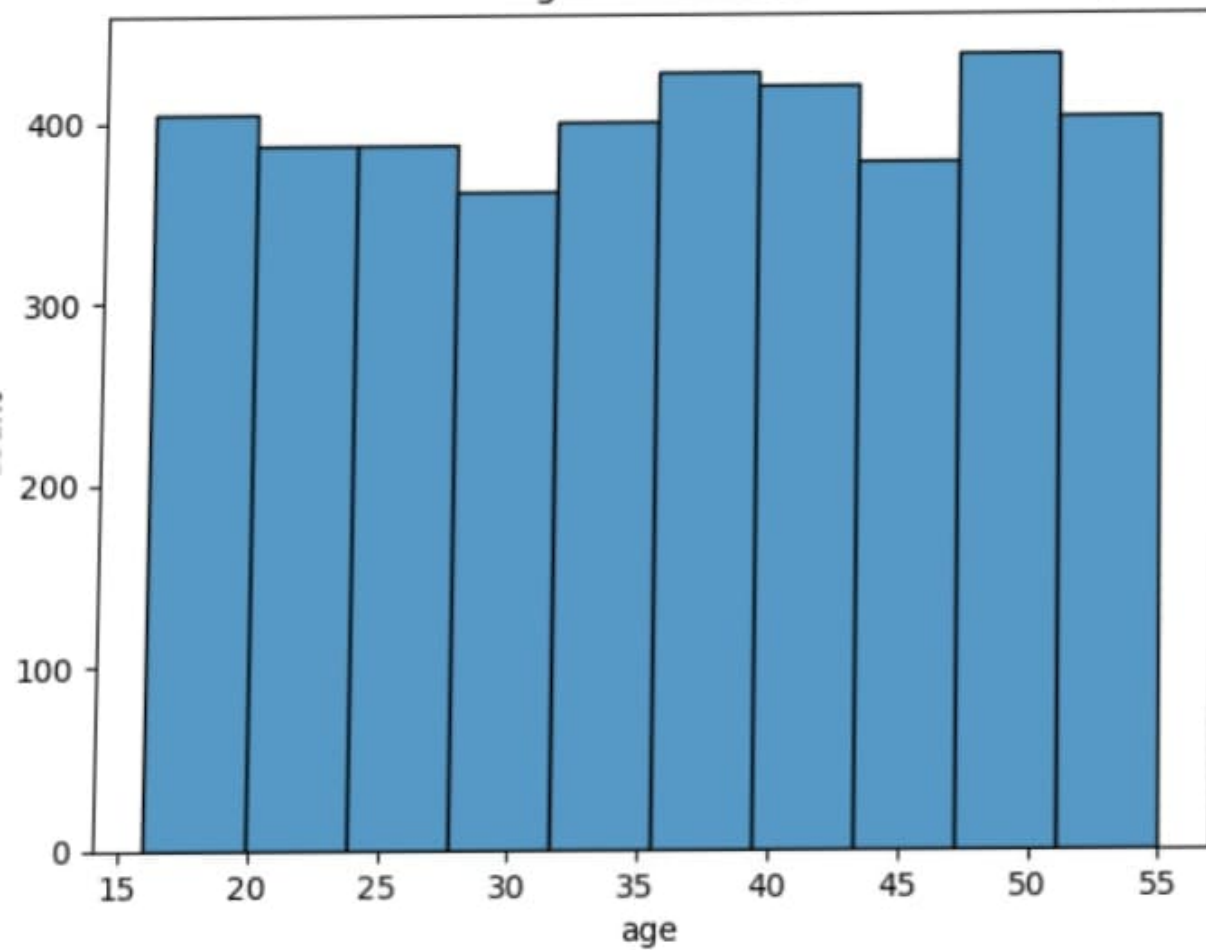
```
] : import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv("online_learning_2026.csv")

sns.histplot(
    data=df,
    x='age',
    bins=10
)

plt.title("Age Distribution")
plt.show()
```

Age Distribution



Analysis & Business Insights:

- Shows the distribution of a single variable.
- Helps identify the most common groups and understand student demographics for better planning.

Bivariate Analysis (Two Variables)

Scatter Plot

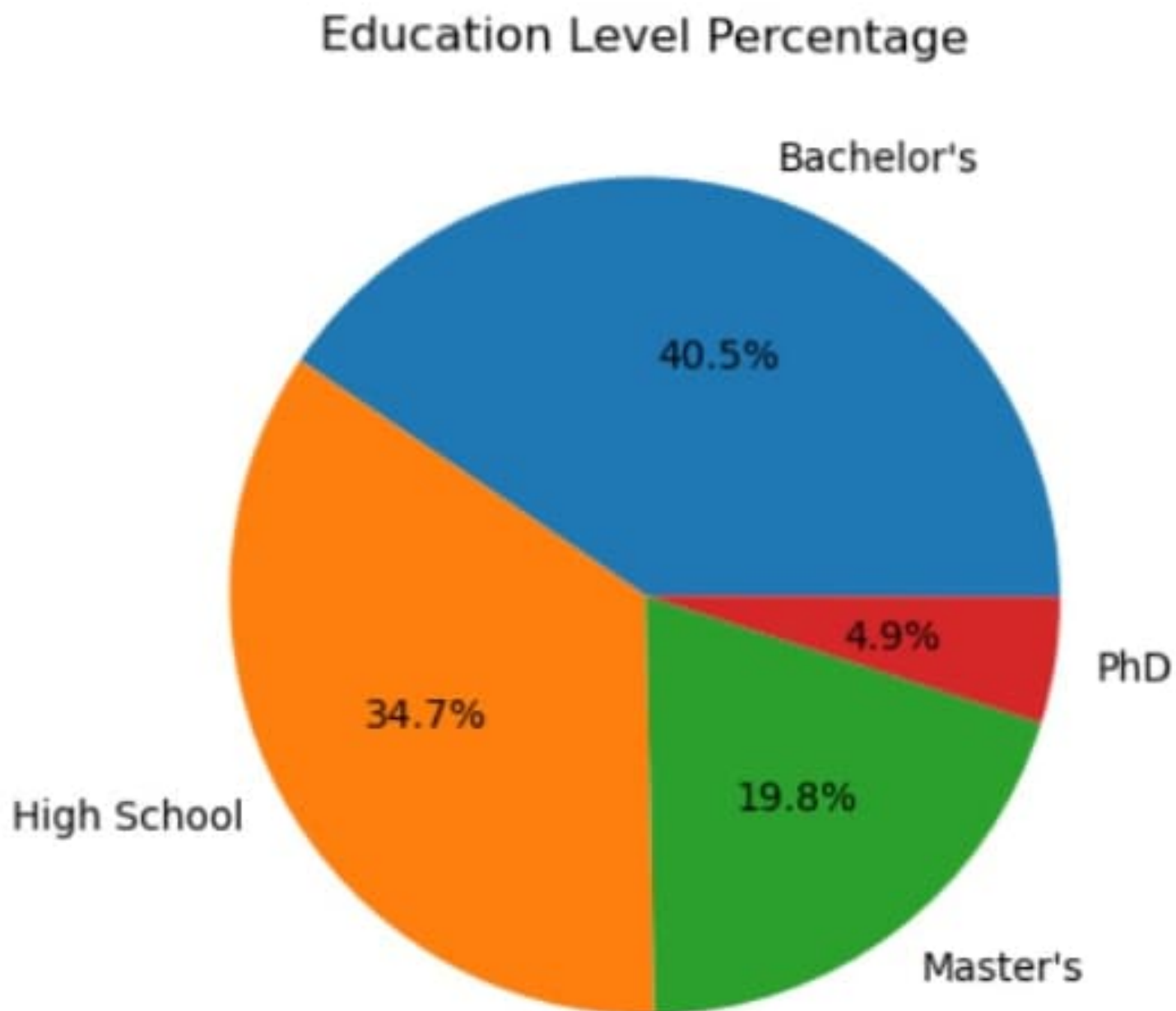
```
import seaborn as sns
import matplotlib.pyplot as plt

sns.scatterplot(
    x='hours_per_week',
    y='quiz_scores_avg',
    data=df
)

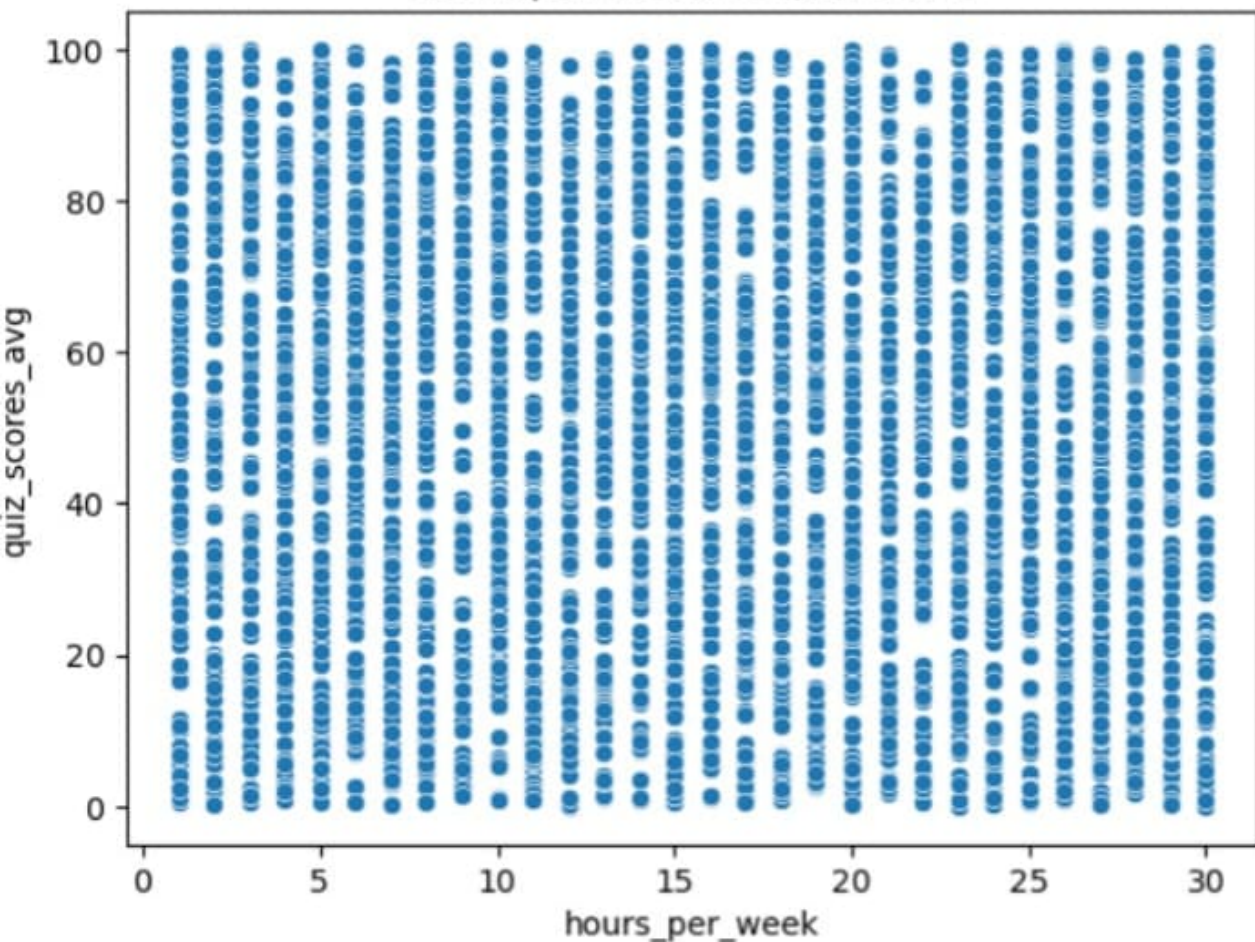
plt.title("Hours per Week vs Quiz Score")
plt.show()
```

Pie chart

```
2]: df['education_level'].value_counts().plot(  
    kind='pie',  
    autopct='%1.1f%%'  
)  
  
plt.title("Education Level Percentage")  
plt.ylabel("")  
plt.show()
```

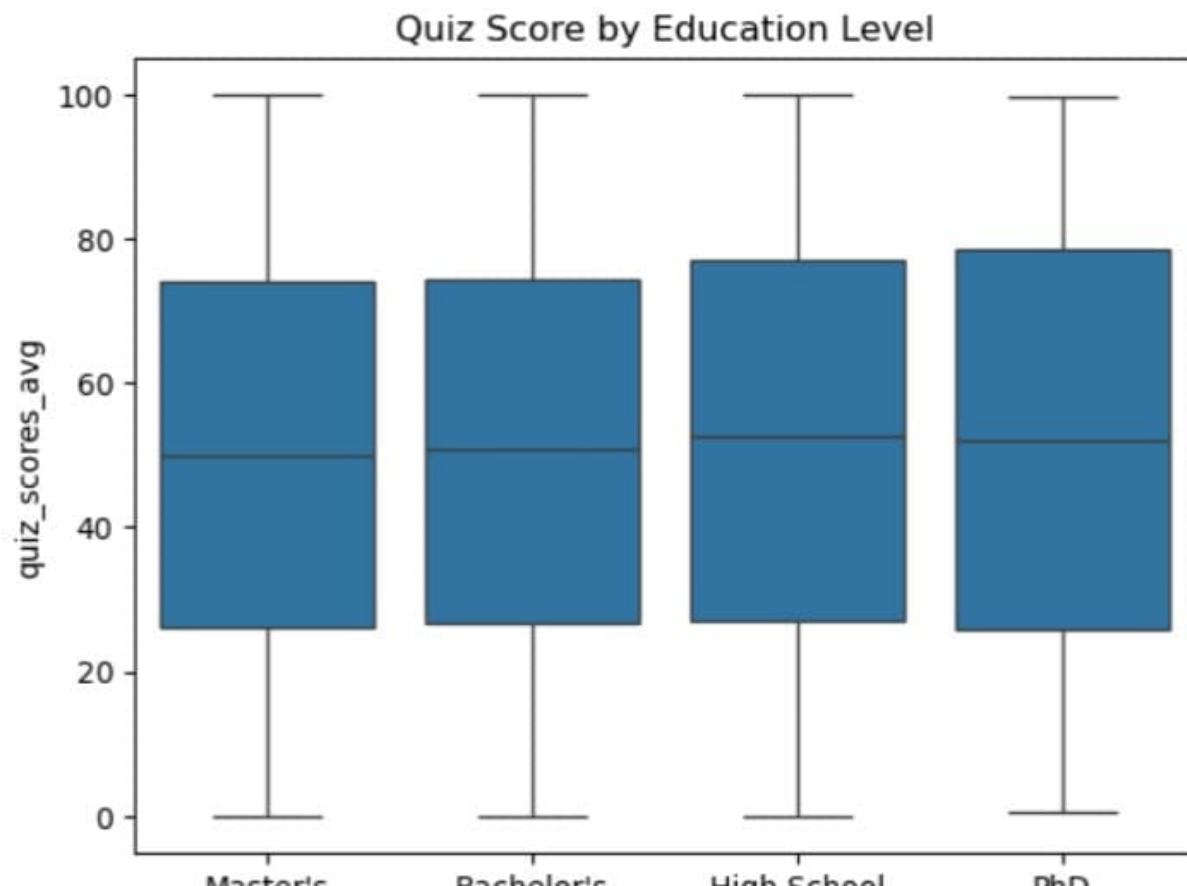


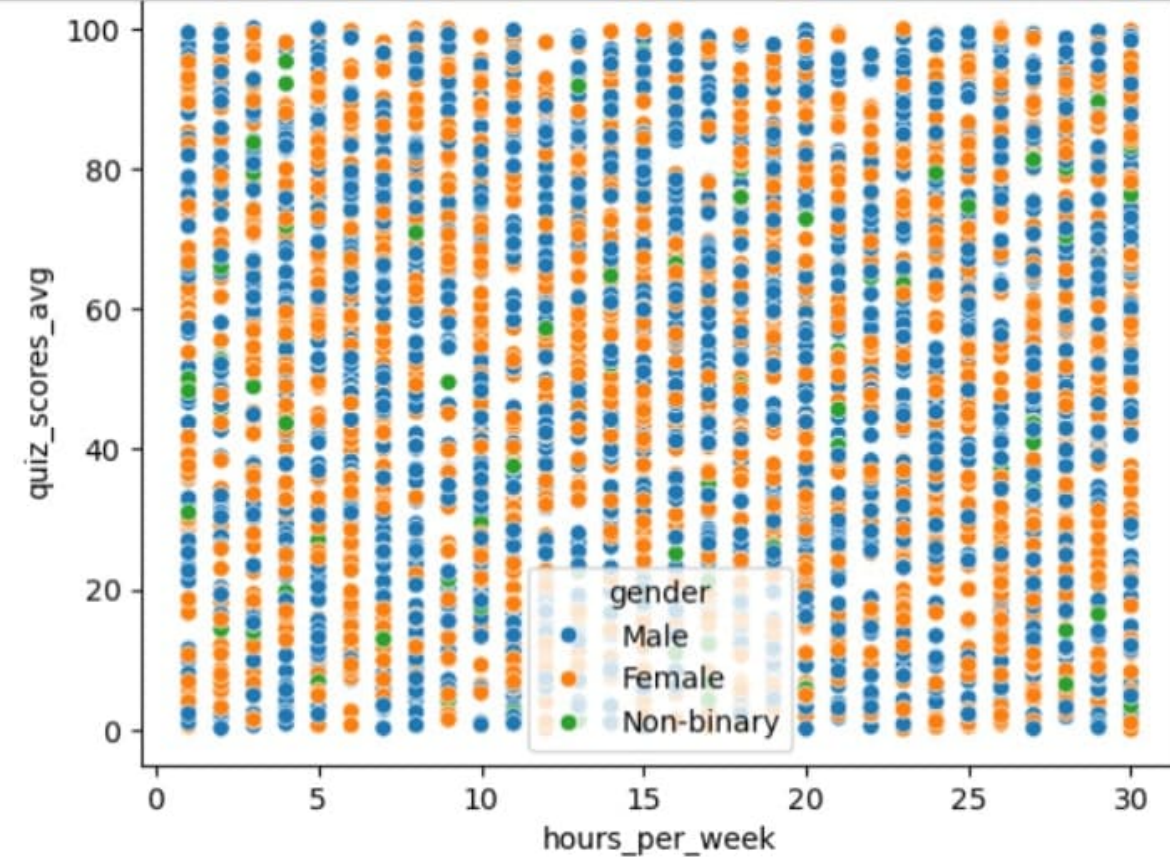
Hours per Week vs Quiz Score



Box plot

```
[4]: sns.boxplot(  
      x='education_level',  
      y='quiz_scores_avg',  
      data=df  
)  
  
plt.title("Quiz Score by Education Level")  
plt.show()
```





Pair plot

```
import seaborn as sns

sns.pairplot(
    df,
    vars=['age', 'hours_per_week', 'quiz_scores_avg'],
    hue='gender'
)
```

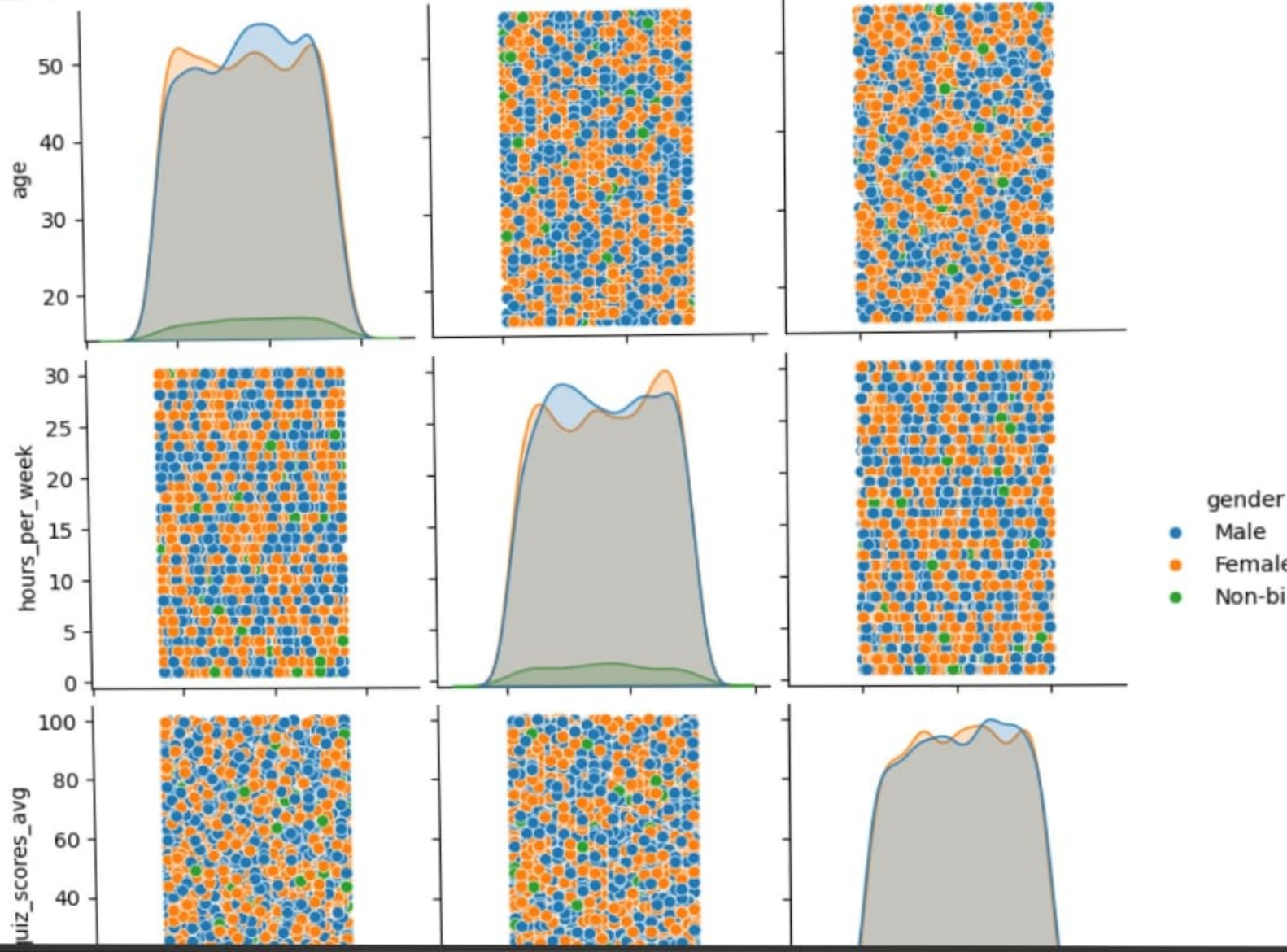

Analysis & Business Insights:

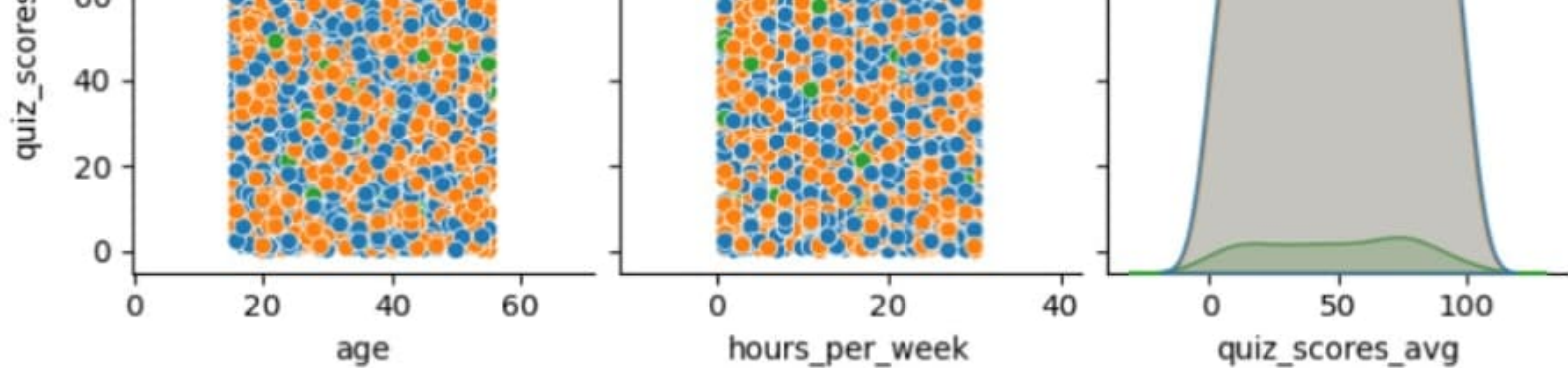
- Shows the relationship between two variables.
- Helps understand how one factor affects another and supports better decision-making.

Multivariate Analysis (More Than Two Variables)

Scatter Plot with Hue

```
sns.scatterplot(  
    x='hours_per_week',  
    y='quiz_scores_avg',  
    hue='gender',  
    data=df  
)  
  
plt.title("Hours vs Quiz Score by Gender")  
plt.show()
```





Analysis & Business Insights:

- Shows relationships among multiple variables.
- Helps identify patterns between different student groups and improve targeting strategies.